

تحليل عميق — نقاط الانهيار المتسلسل

الدوال الأخطر — ANOGS/AnoSDK Anti-Cheat

الدوال التي يؤدي تعطيلها لتعطيل كل الحماية أو كل قنوات الكشف

تقرير دفاعي فقط — بدون محتوى bypass
♦ bypass by Hdanh | ♦ made by @Nopermcc

ملخص تنفيذي — خارطة الانهيار

من بين 10,121 دالة داخلية في البايناري، حددنا 7 دوال تمثل نقاط انهيار متسلسل حقيقية — تعطل أي واحدة منها يُسقط قنوات حماية كاملة أو يُعطّل خط أنابيب الكشف بالكامل.

النتيجة الرئيسية: كل تقارير الكشف (debugger, injection, jailbreak, integrity) تمر عبر نقطة واحدة — دالة sub_0373e8 (report_emit). هذه الدالة تحتوي 4x ret cbz صامت. إذا فشلت = صمت كامل لكل قنوات الكشف.

التصنيف حسب الخطورة — الأخطر أولاً

#1 — sub_0373e8 @ 0x0373e8 — بوابة الإبلاغ الوحيدة (COREREPORT) (Emit)

sec_ancestors=24

ancestors=475

downstream=358 دالة

fan_in=5

SPOF

CRITICAL

لماذا هي الأخطر؟

هذه الدالة هي القناة الوحيدة لإرسال كل نتائج الكشف إلى السيرفر عبر بروتوكول COREREPORT. كل أنظمة الكشف الفرعية — debugger, injection, jailbreak, integrity — تمر عبرها. إذا فشلت = الكشف يعمل محلياً لكنه لا يُبلّغ = السيرفر أعمى.

الدليل من التفكير

```
sub_0373e8:
0x0373f4: cbz x0, #0x37454      ← صامت return → arg0 == NULL : فشل
0x0373fc: cbz x20, #0x37454     ← صامت return → vtable ptr == NULL : فشل
0x037420: cbz x0, #0x37454     ← صامت return → lookup COREREPORT : فشل
0x037438: cbz x0, #0x37454     ← صامت return → channel resolve : فشل

0x037454: ret                ← fallback, لا retry, لا log الخروج الصامت — لا
```

سلسلة الانهيار

→ (register) **sub_0371e0** → (emit) **sub_0373e8** → (router) **sub_0376a0**
→ **COREREPORT channel** → **السيرفر**

إذا فشل أي cbz من الأربعة → كل نتائج الكشف التالية تُسقط بصمت:

نظام الكشف	الدالة المصدر	هل يتأثر؟
Debugger (P_TRACED)	sub_037004	يتعطل الإبلاغ بالكامل
Injection (dyld)	sub_1f3424	يتعطل الإبلاغ بالكامل
Jailbreak	sub_13a5a4 / sub_6bfe4	يتعطل الإبلاغ بالكامل
Memory Integrity	sub_148a58	يتعطل الإبلاغ بالكامل
ObjC Swizzle	sub_13d250	يتعطل الإبلاغ بالكامل

التحصين:

- إضافة قناة إبلاغ احتياطية (fallback channel) لا تعتمد على COREREPORT vtable
- تحويل كل `cbz → ret` صامت إلى `cbz → error_log + fallback_emit`
- عداد فشل إبلاغ مع إنذار محلي إذا تجاوز الحد
- تخزين التقارير الفاشلة في queue محلي وإعادة إرسالها

COREREPORT (Channel قناة تسجيل — sub_0371e0 @ 0x0371e0 — #2

(Register

ancestors=476

downstream=354 دالة

fan_in=5

SPOF

CRITICAL

kill-switch @ 0x29bc38

لماذا هي خطيرة؟

هذه الدالة تُسجل قناة COREREPORT/tdm_report. تحتوي آلية تعطيل ذاتي بعد 3 محاولات فاشلة — عداد عند 0x29bc3c يزداد مع كل فشل، وبعد 3 فشلات يُكتب 0 في flag 0x29bc38 مما يُعطّل القناة نهائياً.

الدليل من التفكيك

```
; --- مسار الفشل في sub_0371e0 ---
0x0372ac: cbz x0, #0x373a4      ← فشل COREREPORT lookup
0x0372c4: cbz x0, #0x373a4      ← فشل channel resolve
0x0372e0: cbz x0, #0x373a4      ← فشل create context

; --- آلية الموت البطيء عند 0x373a4 ---
0x0373a8: ldr w9, [x8, #0xc3c]    ← قراءة عداد الفشل من 0
0x0373ac: add w10, w9, #1         ← ++عداد
0x0373b0: str w10, [x8, #0xc3c]    ← تخزين العداد الجديد
0x0373b4: cmp w9, #3              ← هل وصل 3؟
0x0373b8: b.lt #0x373cc           ← أقل من 3: حاول مرة أخرى
0x0373bc: mov x21, #0
0x0373c4: strb wzr, [x8, #0xc38]   ← تعطيل القناة نهائياً = flag = 0 في
```

المشكلة الهيكلية

لا يوجد آلية إعادة تفعيل. بمجرد كتابة wsr في 0x29bc38 ، القناة ميتة حتى إعادة تشغيل التطبيق. أي خطأ شبكة مؤقت (3 محاولات سريعة) يُعطّل كل الإبلاغ نهائياً.

التحسين:

1. استبدال العداد الخطي بـ exponential backoff مع حد أقصى (لا تعطيل نهائي أبداً)
2. إضافة مؤقت إعادة تفعيل: بعد 60 ثانية من التعطيل، إعادة المحاولة
3. تسجيل حدث تعطيل القناة كحدث أمني (ironically عبر قناة أخرى)
4. إضافة heartbeat: إذا لم تُرسل تقارير لمدة X ثانية، إنذار محلي

#3 — 0x0376a0 @ sub_0376a0 — الموزّع الرئيسي (Router 7-Slot)

slot gate table @ 0x29bc10-7

downstream=364 دالة

fan_in=2

SPOF

CRITICAL

لماذا هي خطيرة؟

هذه هي نقطة الدخول الوحيدة لكل أحداث الأمان بعد التهيئة. كل event ID (من 0x8392 إلى 0x849x) يمر عبر gate table من 7 slots عند 0x29bc10. كل slot = كلمة 32-bit. إذا كان slot = 0 → الحدث يُسَقَط بصمت.

الدليل من التفكير

```
sub_0376a0:
0x0376bc: cmp    w20, #6          ← event_id > 6? → خروج
0x0376c0: b.hi   #0x376d4          ← خروج صامت
0x0376c8: add    x9, x9, #0xc10    ← gate table عنوان = 0x29bc10
0x0376cc: ldr    w8, [x9, w20, uxtw #2] ← slot[event_id] قراءة
0x0376d0: cbz    w8, #0x376e4      ← slot == 0 → ret صامت
0x0376d4: ret                     ← processing خروج - لا
```

سلسلة الاعتماد

هذا الموزّع هو الأب المباشر لـ:

الدالة الابن	الوظيفة	fan_in الخاص
sub_0371e0	تسجيل قناة COREREPORT	5
sub_0373e8	إرسال تقرير COREREPORT	5
sub_0f97fc	subsystem init #1	18
sub_0faa28	subsystem init #2	8
sub_0fbd84	subsystem init #3	15
sub_0fd684	subsystem init #4	9
sub_0fe434	subsystem init #5	16
sub_0fef9c	subsystem init #6	10
sub_0ff308	subsystem init #7	14

التحصين:

1. جعل gate table في ذاكرة read-only بعد التهيئة (vm_protect)
2. فحص سلامة gate table دورياً (CRC32 check)
3. إضافة logging لأحداث مُسَقَّطة (dropped events counter)
4. fallback path: إذا كان slot صفر بشكل غير متوقع → إعادة تهيئة

#4 — sub_037004 @ 0x037004 — كاشف (Fail-Open) Debugger

يستخدم: sysctl, getpid

fan_in=1

FAIL-OPEN

لماذا هي خطيرة؟

هذه الدالة تفحص وجود debugger عبر sysctl(CTL_KERN, KERN_PROC, KERN_PROC_PID) ثم تتحقق من flag P_TRACED. **المشكلة:** إذا فشل استدعاء sysctl نفسه (رجع -1)، الدالة تُرجع 0 = "نظيف" لا debugger.

الدليل من التفتيك

```
sub_037004:
0x03706c: bl    _sysctl          ← sysctl استدعاء
0x037070: cmn   w0, #1           ← هل رجع 1- (فشل) ؟
0x037074: cset  w8, ne           ← w8 = (result != -1) ? 1 : 0
0x037078: ldr   w9, [sp, #0x40]   ← قراءة kp_proc.p_flag
0x037094: and   w9, w9, #0x800    ← عزل P_TRACED bit
0x037098: and   w0, w8, w9, lsr #11 ← w0 = success_flag AND p_traced

← P_TRACED بغض النظر عن w0=0 → w8=0 : فشل sysctl إذا
← FAIL-OPEN: فشل sysctl = "لا debugger" هذا
```

خطورة Fail-Open

الاستغلال بسيط من الناحية الدفاعية: أي خطأ في (sandbox restriction, resource limit) sysctl يُعطّل كشف الdebugger. التصميم الصحيح هو Fail-Closed: أي فشل = "مشتبّه".

التحصين:

1. تغيير المنطق إلى Fail-Closed: إذا فشل sysctl → إرجاع 1 (مشتبّه)
2. إضافة فحص ثانٍ مستقل (task_info TASK_DEBUG_INFO_INTERNAL)
3. مراقبة تكرار فشل sysctl (أكثر من مرتين = حالة مشبوهة)
4. إرسال تقرير فشل sysctl نفسه كحدث أمني

#5 — sub_0a1414 @ 0x0A1414 — بوابة Syscall الخام

sec_ancestors=30

ancestors=540

fan_in=9

SPOF

لماذا هي خطيرة؟

هذه الدالة هي بوابة الsyscall المباشرة التي تتجاوز libc لمنع hooks. تستخدمها 9 دوال (send, recv, connect وغيرها). هي 13 تعليمة فقط — سطح هجوم صغير جداً لكن بدون أي حماية ذاتية.

الدليل من التفكير

```
sub_0a1414:
0x0a1414: ldp x1, x2, [sp]      ← stack من ال args تحميل
0x0a1418: ldp x3, x4, [sp, #0x10]
0x0a141c: ldp x5, x6, [sp, #0x20]
0x0a1420: ldr x7, [sp, #0x30]
0x0a1424: mov x16, #0          ← syscall number = 0 (placeholder)
0x0a1428: svc #0x80             ← المباشر syscall ال
0x0a142c: b.lo #0xa1444         ← نجاح → skip error handler
0x0a1438: bl sub_0a13ec         ← معالج الخطأ
```

التبعيات

9 دوال تعتمد عليها — أهمها sub_0f8210 (send) و sub_0f829c (recv). Flag عند 0x2ab430 (bit 13/14) يُحدد هل يستخدم libc أو syscall مباشر. هذا Flag نفسه = نقطة حرجة.

التحصين:

1. تشفير/تشويش عنوان بوابة ال (anti-pattern scanning) syscall
2. فحص سلامة التعليمات في البوابة (self-integrity check)
3. حماية Flag 0x2ab430 ب vm_protect read-only بعد التهيئة
4. إضافة بوابة syscall احتياطية بعنوان مختلف (redundancy)

#6 — AnoSDKInit @ 0x0F0FD0 sub_0f0fd0 (نقطة دخول التهيئة)

return مضلل

downstream=383 دالة

fan_in=0 (export)

SPOF

لماذا هي خطيرة؟

هذه هي نقطة دخول SDK الوحيدة. المضيف (اللعبة) يستدعيها مرة واحدة. المشكلة: القيمة المرجعية لا تعكس نجاح التهيئة — بل تُحسب من ال argument الأول!

الدليل من التفكير

```
sub_0f0fd0:
0x0f0fdc: mov x19, x0          ← حفظ arg0
0x0f0fe0: bl sub_02d69c         ← استدعاء init_backend
0x0f0fe4: mvn w8, w19          ← w8 = NOT(arg0)
0x0f0fe8: lsr w0, w8, #0x1f     ← w0 = bit31 of NOT(arg0)
0x0f0ff8: b sub_0376a0         ← إلى الموزع tail-call

; sub_02d69c (init_backend):
0x02d6bc: mov w0, #0           ← !يرجع دائماً 0
0x02d6c8: ret
```

المشكلة المزروجة

1. sub_02d69c يرجع دائماً 0 — لا يوجد فحص نجاح/فشل
2. Return value لل AnoSDKInit = 31 >> NOT(arg0) — مشتق من المدخل، لا من نتيجة التهيئة
3. المضيف لا يستطيع التمييز بين تهيئة ناجحة وفاشلة

التحصين:

1. إرجاع قيمة حقيقية من init_backend تعكس نجاح/فشل كل مكوّن
2. فحص post-init: بعد التهيئة، التحقق أن كل القنوات مُسجّلة وفعّالة
3. إضافة callback للمضيف في حالة فشل التهيئة المتأخر
4. heartbeat أولي: أول تقرير خلال 5 ثوانٍ من التهيئة أو إنذار

#7 — sub_102d48 @ 0x102D48 — مُصنّف الأحداث (79) — Event Switch

(نوع)

event type 79

downstream=1962 دالة

fan_in=1

SPOF

لماذا هي خطيرة؟

هذه الدالة تُصنّف 79 نوع حدث أمني عبر jump table واحدة عند 0x2656fc . fan_in=1 يعني أنها تُستدعى من مكان واحد فقط (sub_11af80). إذا تعطلّ هذا المكان الوحيد = 79 نوع كشف يتوقف.

الدليل من التفكيك

```
sub_102d48:
0x102d84: bl    sub_10298c      ← فحص صلاحية الحدث
0x102d88: tbnz  w0, #0, #0x102d90 ← valid → continue
0x102d8c: b     #0x102dc8      ← invalid → DROP صامت

0x102de8: cmp   x8, #0x4e      ← event_id > 78?
0x102df0: b.hi  #0x103d1c      ← DROP صامت → خارج النطاق

0x102df8: add   x8, x8, #0x6fc  ← jump table @ 0x2656fc
0x102e00: ldrsw x10, [x8, x11, lsl #2] ← offset قراءة ال
0x102e08: br    x9              ← القفز لمعالج الحدث

; أمثلة أحداث:
; "ms_data_crc"      → mrpcs_data_crc_error
; "ms_data_len"      → mrpcs_data_len_error
; "ms_data_mod_len"  → mrpcs_data_mode_name_len_error
```

79 نوع حدث — عبر نقطة واحدة

كل أحداث النزاهة والتلاعب تمر عبر هذا المفرّع الوحيد. Drop صامت لأي حدث = لا خطأ، لا log، لا retry.

التحصين:

1. إضافة مُصنّف احتياطي يتحقق من سلامة الأساسي (watchdog pattern)
2. حماية jump table 0x2656fc بـ read-only memory
3. عداد أحداث مُسقطة مع إنذار إذا تجاوز الحد
4. إضافة caller ثانٍ لـ sub_102d48 (redundant invocation path)

خلاصة — خارطة السيطرة المركزية

تأثير التعطيل

نوع الثغرة

الخطورة

الدالة

1	sub_0373e8	حرج	SPOF + Silent Drop	صمت كامل لكل قنوات الكشف
2	sub_0371e0	حرج	Self-Kill + SPOF	تعطيل قناة COREREPORT نهائياً بعد 3 فشلات
3	sub_0376a0	حرج	SPOF + Gate Table	إسقاط كل الأحداث الأمنية (364 دالة downstream)
4	sub_037004	عالي	Fail-Open	إخفاء وجود debugger
5	sub_0a1414	عالي	SPOF + No Self-Check	تعطيل النقل المضاد للـ (540 ancestor) hook
6	sub_0f0fd0	عالي	Blind Init + Fake Return	تهيئة فاشلة غير مكتشفة
7	sub_102d48	عالي	SPOF + Silent Drop	إسقاط 79 نوع حدث أمني (1962 دالة downstream)

العناوين الحرجة في DATA__ — المفاتيح العالمية

العنوان	النوع	الوظيفة	الخطورة
0x29bc10	u32 × 7	Gate table — 0 = drop event	كتابة 0 = تعطيل فئة كاملة
0x29bc38	u8	Channel enable flag	كتابة 0 = قتل COREREPORT
0x29bc3c	u32	Retry counter	تضخيم = تسريع التعطيل الذاتي
0x2ab430	u32	Transport flag (bit 13/14)	تغيير = تحويل من syscall إلى libc hookable
0x2656fc	s32 × 79	Event jump table	تعديل = إسقاط أنواع كشف محددة
0x290580	target × 634	CFF successor table	تعديل = تعطيل منطق CFF

التوصيات الاستراتيجية

1. القضاء على نقاط الفشل الوحيدة (SPOF Elimination)

كل الثغرات السبع تشترك في نمط واحد: مسار واحد بلا بديل. الحل الأساسي:

- قناة إبلاغ ثانوية مستقلة عن COREREPORT vtable
- مُصنَّف أحداث ثانوي يتحقق من الأساسي
- بوابة syscall احتياطية

2. تحويل Fail-Open إلى Fail-Closed

كل دالة كشف يجب أن تعامل الفشل كـ "مشتببه" وليس "تنظيف".

3. حماية الحالة العالمية

العناوين الستة في DATA__ هي "مفاتيح قتل" — يجب حمايتها بـ:

- vm_protect(VM_PROT_READ) بعد التهيئة
- فحص سلامة دوري (integrity polling)
- نسخ mirror في مواقع عشوائية

4. إزالة الصمت

كل `cbz → ret` صامت يجب أن يُحوَّل إلى حدث مُسجَّل. الصمت هو العدو الأول لأنظمة الأمان.